

ZERO_BLUFF_USER_MANUAL

HelixCode User Manual — Zero-Bluff Phase 5 Edition

Audience: End users installing and using HelixCode. **Companion:** For depth, see [README.md](#) (3,240-line comprehensive reference), [INDEX.md](#), and [SUMMARY.md](#). **Scope:** This document covers the 30 shipped features (F01–F30) and the Zero-Bluff infrastructure with end-user-actionable detail. Every command shown is real and was validated against the codebase. **Last updated:** 2026-05-12

1. Getting Started

1.1 System Requirements

- Linux (kernel ≥ 5.10), macOS (≥ 12), or Windows (10+ via WSL2)
- Go 1.26 (only for builds from source; binary releases need no toolchain)
- PostgreSQL 15+ and Redis 7+ for full-feature mode (single-user mode runs without them)
- 4 GB RAM minimum, 16 GB recommended for repomap / large codebases
- Optional: Ollama for local LLM (default offline provider)

1.2 Install From Source

```
git clone git@github.com:HelixDevelopment/HelixCode.git
cd HelixCode
./setup.sh
    # initialises submodules, builds binaries, runs smoke
    test
```

`setup.sh` performs (in order):

1. `git submodule update --init --recursive`
2. Verifies Go 1.26 toolchain
3. `cd HelixCode && make build` → produces `bin/helixcode` (server) and `bin/cli`
4. Runs `make verify-compile smoke test`

1.3 First-Run Wizard (F01 + F02)

```
./helix_code/bin/cli wizard
```

The wizard walks through:

- Selecting your primary LLM provider (Anthropic, OpenAI, Ollama, etc.)
- Pasting API keys (stored in `~/ .config/helixcode/api_keys.sh`, mode 0600)
- Choosing an approval mode (see §3.1)
- Configuring sandbox profile (see §3.2)

After completion you have a populated `~/ .config/helixcode/config.yaml`.

1.4 Configuration Layers

Resolution order (highest precedence first):

- 1. CLI flag (`--approval`, `--config`, `--provider`, ...)
- 2. Environment variable (`HELIXCODE_APPROVAL`, `HELIXCODE_PROVIDER`, ...)
- 3. User config file (`~/ .config/helixcode/config.yaml`)
- 4. Repo-local override (`.helixcode.yaml` in current dir)
- 5. Built-in defaults

2. Core Features by Category

2.1 Editing & Filesystem (F03, F17, F26)

Feature	What it does	How to invoke
F03 — Tool result persistence	Every tool call writes structured results to <code>~/ .local/share/helixcode/sessions/<id>/tool_results.jsonl</code>	Automatic on every edit
F17 — Smart File Editing	4 formats (diff/whole/search-replace/line) auto-selected per LLM model	Used internally by the edit tool
F26 — Workspace Manager	Tracks the active project root	<code>/workspace show</code> , <code>/workspace switch <path></code>

Smart-edit format selection happens automatically based on the model’s strengths; see `helix_code/internal/editor/README.md`.

2.2 Approval & Sandbox (F02, F14, F21)

Feature	Modes / Profiles
F02 — Permission rules	Allow / deny tool, allow / deny path
F14 — Sandboxed Shell Execution	read-only, workspace-write, danger-full-access
F21 — Codex Approval Modes	suggest, auto-edit, full-auto, dangerously-bypass

Set the mode at startup:

```
./helix_code/bin/cli --approval suggest
./helix_code/bin/cli --approval auto-edit
```

```
./helix_code/bin/cli --approval full-auto          # requires sandbox
HELIXCODE_APPROVAL=full-auto ./helix_code/bin/cli
```

Inspect / change at runtime:

```
/approval status      # shows current mode, source, descriptors
/approval set auto-edit
/approval show        # explains the 4x4 matrix
```

full-auto mode requires sandbox enforcement (approval: full-auto mode requires sandbox); the executor refuses to start without it.

2.3 Sessions, Memory & Plans (F07, F08, F11, F24, F25)

Feature	Slash command / CLI
F07 — Background Task System	task subcommand; tracks long-running jobs with checkpoints
F08 — Plan Mode	/plan on, /plan off; agent must produce a plan before edits
F11 — Session Transcript & Resume	./bin/cli sessions list, ./bin/cli sessions resume <id>
F24 — Project Memory	/memory show, /memory add <fact>, /memory clear
F25 — Plandex Plan Trees	/plan tree, /plan rollback <node>

Project memory loads .helixcode/memory.md from the repo root and caches it across sessions.

2.4 LLM Providers (F12)

15 providers ship in internal/providers/: Anthropic Claude, OpenAI, Google Gemini, AWS Bedrock, Azure OpenAI, Google VertexAI, Groq, Mistral, DeepSeek, xAI, OpenRouter, Ollama, llama.cpp, LiteLLM gateway, CharacterAI.

There are TWO provider-access paths (anti-bluff: an earlier revision of this doc claimed ./bin/cli --provider ollama worked directly; in reality only 4 providers route through the F12 CLI shortcut, and the others go through the server-mediated path):

Path A — F12 direct-cloud CLI shortcut (15 providers as of round 41 final):

```
./bin/cli --provider anthropic --model claude-3-5-sonnet
./bin/cli --provider bedrock   --model anthropic.claude-3-5-sonnet
./bin/cli --provider vertexai  --model gemini-1.5-pro
./bin/cli --provider azure     --model gpt-4o
./bin/cli --provider groq      --model llama-3.3-70b-versatile
./bin/cli --provider openai    --model gpt-4o
./bin/cli --provider gemini     --model gemini-1.5-pro
```

```

./bin/cli --provider openrouter --model openai/gpt-oss-20b:free # auto-selects from live /models
./bin/cli --provider xai --model grok-3-fast-beta
./bin/cli --provider qwen --model qwen-max
./bin/cli --provider copilot --model gpt-4o
./bin/cli --provider mistral --model mistral-small-latest
./bin/cli --provider deepseek --model deepseek-chat
./bin/cli --provider ollama --model llama3.2
# local — needs ollama running
./bin/cli --provider llamacpp --model <path-to-model>
# local — needs llama-server running

```

These fifteen read credentials from the user’s ~/.config/helixcode/ or HELIX_LLM_PROVIDER env (e.g. GROQ_API_KEY, OPENAI_API_KEY, GEMINI_API_KEY, XAI_API_KEY, OPENROUTER_API_KEY, QWEN_API_KEY, MISTRAL_API_KEY, DEEPSEEK_API_KEY, GITHUB_TOKEN for Copilot; local providers ollama/llamacpp default to their standard ports :11434 / :8080 on localhost and need no API key). They construct the provider directly in the CLI process — no server required. This is the **just-plug-in-an-API-key-and-go** path for the most-used cloud providers AND the most-used local providers, giving HelixCode CLI parity with modern single-binary CLI agents (Claude Code, Aider, Cline) for these fifteen.

Path B — server-mediated (the remaining 3 providers):

For vLLM, LocalAI, LM Studio (and any other OpenAI-compatible passthrough endpoint with no dedicated constructor): add an entry under llm.providers: in helix_code/config/config.yaml, start the HelixCode server (make build && ./bin/helixcode server), and access the provider via the server’s REST API or via the CLI’s -server-url flag pointing at the server. The server hosts the provider manager (CONST-039) and exposes a unified API. These providers are served via the OpenAI-compatible passthrough since they don’t have dedicated standalone constructors.

A ./bin/cli --provider <Path-B-provider> invocation surfaces a directed error (since round 41) that names this path explicitly — no more ambiguous “unknown cloud provider” message.

List available models (queries every configured provider — no hardcoding per CONST-036):

```

./bin/cli --list-models # CLI flag — shows F12 + verifier-known providers
# Or, inside the interactive REPL:
> models

```

2.5 Tools & Extensibility (F05, F06, F09, F10, F15, F30)

Feature	Description
F05 — Hooks	Lifecycle hooks (pre-tool, post-tool, on-error, on-commit) — Go plugins or shell scripts in ~/.config/helixcode/hooks/
F06 — MCP Full Lifecycle	Model Context Protocol servers managed via /mcp commands

Feature	Description
F09 — Slash Commands	Built-in + user-defined; see /help
F10 — Skill System	Skills as .md + frontmatter in ~/.config/helixcode/skills/
F15 — Subagent Team	Dispatch parallel subagents via the Task tool
F30 — Continue IDE	IDE companion integration

2.6 Git & Repo (F04, F22, F27)

Feature	Slash command
F04 — Git Worktree Agent Isolation	/worktree create <branch>, /worktree exit
F22 — Aider Git Auto-Commit	/git_auto_commit on, /git_auto_commit off
F27 — Aider Voice & Repomap	/repomap show, /voice on (requires audio backend)

Auto-commit produces LLM-summarised messages with a Co-Authored-By trailer and runs a secret filter on the commit subject.

2.7 Browser & Voice (F23, F27)

```
/browser navigate https://example.com
/browser snapshot
/browser click <selector>
/voice on                # mic capture → transcription → prompt
```

2.8 UI & Theming (F18, F19, F20)

Feature	Slash command
F18 — No-Flicker Rendering	Automatic; tune via ~/.config/helixcode/render.yaml
F19 — Ask User Question	LLM-generated, locale-aware (CONST-046) — never hardcoded
F20 — Theme System	/theme list, /theme set <name>, /theme reload

2.9 Refactoring & Code Intelligence (F13, F28, F29)

Feature	Slash command / capability
F13 — LSP Integration	./bin/cli lsp list-servers, /lsp start <lang>, /lsp hover
F28 — Kilocode Refactoring	/refactor rename <sym>, /refactor extract, /impact <sym>

Feature	Slash command / capability
F29 — RooCode Full Port	/roo agent <task>, end-to-end agent workflows

2.10 Telemetry & Observability (F16)

```
HELIXCODE_OTEL_ENDPOINT=http://localhost:4317 ./bin/cli
/telemetry status
/telemetry flush
```

Spans, metrics, and logs are exported via OpenTelemetry (gRPC default). See `docs/deployment_guide/README.md §3`.

3. CLI Reference (summary)

Full reference: [docs/COMPLETE_CLI_REFERENCE.md](#) (1,031 lines).

3.1 Top-level subcommands

Command	Purpose
helixcode wizard	First-run setup
cli	Interactive REPL
cli --list-models	Enumerate models from every configured provider
cli sessions list	Show past sessions
cli sessions resume <id>	Resume a transcript
cli lsp list-servers	Show configured LSP servers
cli task list	Active background tasks
helix-config	Standalone config validator

3.2 Common flags

Flag	Env var	Default
--approval	HELIXCODE_APPROVAL	suggest
--provider	HELIXCODE_PROVIDER	ollama
--model	HELIXCODE_MODEL	provider default
--config	HELIXCODE_CONFIG	~/.config/helixcode/config.yaml
--workspace	HELIXCODE_WORKSPACE	\$PWD
--no-sandbox	HELIXCODE_NO_SANDBOX	false (refused with full-auto)

3.3 Built-in slash commands

/help, /approval, /git_auto_commit, /memory, /plan, /worktree, /workspace, /browser, /voice, /refactor, /impact, /lsp, /mcp, /theme, /telemetry, /roo, /repomap, /sessions, /skills, /hooks.

User-defined commands live in ~/.config/helixcode/commands/*.md (markdown + frontmatter).

4. Troubleshooting

For comprehensive troubleshooting see [docs/COMPLETE_TROUBLESHOOTING_GUIDE.md](#) (1,174 lines).

Common errors:

Symptom	Likely cause	Fix
approval: full-auto mode requires sandbox	Sandbox disabled while in full-auto	Drop --no-sandbox or change mode
provider not configured: <name>	API key missing	Run ./bin/cli wizard or set <NAME>_API_KEY
git_auto_commit: working tree clean	No mutations to commit	Expected no-op
lsp: server not found for <lang>	Server binary missing	./bin/cli lsp install <lang>
mcp: handshake failed	Server crashed before init	Check ~/.local/share/helixcode/mcp/<name>.log

5. Configuration File Reference

~/.config/helixcode/config.yaml:

```
approval:
  mode: suggest # suggest|auto-edit|full-auto|
    dangerously-bypass
  config_file: ~/.config/helixcode/approval.yaml
sandbox:
  profile: workspace-write # read-only|workspace-write|
    danger-full-access
providers:
  default: anthropic
  anthropic:
    model: claude-3-5-sonnet
session:
```

```
retention_days: 30
memory:
  path: .helixcode/memory.md
telemetry:
  enabled: true
  endpoint: http://localhost:4317
```

See [docs/COMPLETE_CONFIGURATION_DOCUMENTATION.md](#) (1,150 lines) for every field.

6. Anti-Bluff Guarantee (for users)

Every visible feature is backed by:

- A passing test (`helix_code/<pkg>/*_test.go`).
- A Challenge that exercises the end-to-end user workflow against real infrastructure (`challenges/`).
- An evidence trail in `docs/improvements/PROGRESS.md`.

If a feature does not behave as documented, file an issue with the failing command — per CONST-035 documentation mismatches are critical defects.

Sources verified 2026-05-29: <https://go.dev/doc/devel/release>, <https://www.postgresql.org/support/versioning/>, <https://github.com/redis/redis/releases>, <https://docs.ollama.com/faq>, <https://opentelemetry.io/docs/specs/otlp/>, <https://platform.claude.com/docs/en/docs/build-with-claude/prompt-caching>

Cross-referenced every external version pin / port / service reference in this operator manual against the latest official docs per Constitution §11.4.99:

- **Go 1.26** (§1.1, §1.2) — **CONFIRMED current.** `go.dev` release history shows Go 1.26.0 (2026-02-10), latest patch **go1.26.3** (2026-05-07). Matches `CLAUDE.md` §3.1. No change.
- **PostgreSQL 15+** (§1.1) — **CONFIRMED valid.** `postgresql.org` versioning page: 15 is supported until 2027-11; 18 is the latest major (18.4). The 15+ floor correctly admits 15–18. No change.
- **Redis 7+** (§1.1) — **CONFIRMED valid.** `github.com/redis/redis/releases`: 7.4.x and 7.2.x still receive security updates and Redis 8.x is GA; the 7+ floor is correct. No change.
- **Ollama default : 11434** (§2.4, F12 local path) — **CONFIRMED.** `docs.ollama.com/faq`: “Ollama binds 127.0.0.1 port 11434 by default” (overridable via `OLLAMA_HOST`). No change.
- **OpenTelemetry OTLP gRPC : 4317** (§2.10, §3.2) — **CONFIRMED.** `opentelemetry.io` OTLP spec: default OTLP/gRPC port is 4317 (HTTP is 4318). No change.
- **macOS ≥ 12 / Windows 10+ via WSL2 / Linux kernel ≥ 5.10** (§1.1) — these are HelixCode’s own declared minimum-support floors (see `docs/platforms/`), not a single vendor’s “latest” claim; the supported-platforms manifest (`docs/platforms/`

supported_platforms.yaml) is the source of truth. No third-party-doc contradiction found.

CONST-036 model-ID notes (flagged, NOT re-pinned): the §2.4 / §5 example model identifiers (claude-3-5-sonnet, gemini-1.5-pro, gpt-4o, llama-3.3-70b-versatile, grok-3-fast-beta, qwen-max, mistral-small-latest, deepseek-chat, llama3.2, openai/gpt-oss-20b:free) and the anthropic.model: claude-3-5-sonnet config snippet are **LLMsVerifier-runtime-sourced** — they are illustrative --provider/--model invocation examples and may be stale. The authoritative live list is ./bin/cli --list-models/helixcode llm models list, which queries every configured provider per CONST-036. Not re-pinned per CONST-036 (LLMsVerifier is the single source of truth). The provider-count claims (“15 providers”, “Path B — remaining 3”) describe HelixCode’s own integration surface, not a vendor figure, and were not externally verifiable.

Negative findings: no Anthropic/OpenAI/etc. account-safety or login-flow instructions are present in this manual that would carry a ban-risk under §11.4.99(D); the only credential guidance is local .env / api_keys.sh (mode 0600) handling, which contradicts no vendor policy. OpenAI’s reasoning-guide page was not fetched here (it returns HTTP 403 to automated fetch — see the FEATURE_IMPLEMENTATION_COMPLETE footer); this manual cites no OpenAI URL.